IC-307D: DATA STRUCTURES & ALGORITHMS LAB

COMPREHENSIVE LABORATORY REPORT & SYSTEM ARCHITECTURE MANUAL

Design, Theoretical Analysis, and Pure-Python Implementation of Foundational Data Structures and Classical Algorithms (Units 1 through 7)

Course Code:	IC-307D
Course Title:	Data Structures and Algorithms Laboratory
Student Identity / Roll:	Jeet Verma / IC-2K23-46
Department:	Department of Computer Science & Engineering
Institution:	International Institute of Professional Studies (IIPS), Devi Ahilya Vishwavidyalaya (DAVV), Indore, Madhya Pradesh, India
Academic Term:	Academic Year 2026–2027
Implementation Language:	Python 3.8+ (Pure Standard Library, Zero Third-Party Data Structure Dependencies)
Units Covered:	Units 1 through 7 (Arrays, Sorting & Searching, Linked Lists, Stacks, Queues, Trees, Graphs)

Notice: *This academic laboratory report documents the actual algorithms, data structures, and application systems implemented in the course repository. All implementation descriptions correspond directly to the available source code. Theoretical complexity analysis is based on established computer science principles. Experimental results and observations are included only where supported by actual execution.*

Table of Contents

Section	Title / Subject Area	Curricular Focus
1	Cover Page & Academic Identification	Course & Student Metadata
2	Table of Contents	Document Structure
3	Introduction to Data Structures and Algorithms	Core Principles & Taxonomy
4	Objectives of the Laboratory	Course Outcomes CO1, CO2, CO3
5	Programming Environment and Tools	Linux, Python Standard Library, Testing
6	Unit 1: Arrays and Matrices	1D/2D Arrays, Sparse Triples, Matrix Calculator
7	Unit 2: Sorting and Searching	7 Sorts, Binary Search, Hash Tables, Applications
8	Unit 3: Linked Lists	SLL, DLL, CLL, Polynomials, Text Buffer, Playlist
9	Unit 4: Stacks	Static/Dynamic Stacks, Infix/Postfix/Prefix, Undo/Redo
10	Unit 5: Queues	Linear/Circular Queues, Deque, Priority Heaps, Scheduling
11	Unit 6: Trees	Traversals, BST, AVL Trees, Expression Trees, Symbol Table
12	Unit 7: Graphs	Matrix/List, BFS, DFS, Dijkstra, Maze, Prerequisites
13	Important Algorithms and Approaches	Algorithmic Paradigms & Invariants
14	Complexity Analysis Summary	Master Asymptotic Reference Table
15	Applications and Practical Use Cases	Operating Systems, Compilers, DBs, Networks
16	Challenges Encountered	Algorithmic & Implementation Pitfalls Solved
17	Learning Outcomes	Competencies & Architectural Insights
18	Conclusion	Synthesis of First-Principles Development
19	References	Academic Literature & Technical Documentation

3. Introduction to Data Structures and Algorithms

A **data structure** is a specialized format for organizing, processing, retrieving, and storing data in computer memory, designed to enable efficient access and modification. An **algorithm** is a finite, well-defined sequence of unambiguous computational instructions that transforms an input into a required output.

The discipline of Data Structures and Algorithms (DSA) forms the intellectual foundation of computer science and software engineering. Computational problems in operating systems, compiler construction, database management systems, artificial intelligence, and network routing depend critically on selecting appropriate structural representations and algorithmic strategies. The choice of data structure directly determines the asymptotic efficiency, cache utilization, and scalability of an application.

Data structures are taxonomically classified into several principal categories:

- **Linear Data Structures**: Elements form a sequential continuum where each member has a unique predecessor and successor (except boundaries). Examples include Arrays, Linked Lists, Stacks, and Queues.
- **Non-Linear Data Structures**: Elements exhibit multi-level hierarchical (Trees) or arbitrary relational (Graphs) interconnections, reflecting complex real-world dependencies.
- **Homogeneous vs. Heterogeneous**: Contiguous arrays store elements of identical data types, whereas composite linked structures can encapsulate diverse payload records.
- **Static vs. Dynamic**: Static structures allocate bounded memory upfront, whereas dynamic structures expand and contract on the system heap at runtime.

To evaluate algorithmic efficacy independently of specific hardware clock speeds, computer scientists utilize asymptotic notations (O, Ω, Θ) to model execution time and auxiliary space complexity as input size $n \to \infty$.

4. Objectives of the Laboratory

The primary objective of the IC-307D Data Structures and Algorithms Laboratory is to cultivate first-principles competency in designing, implementing, empirically analyzing, and applying foundational data structures and classical algorithms in pure Python. The curriculum is mapped to three Core Outcomes (COs):

- **CO1: Linear Data Structures**: Master contiguous memory arrays, 2D matrix transformations, sparse coordinate lists, singly/doubly/circular linked lists, static/dynamic stacks, linear/circular queues, and double-ended queues.
- **CO2: Sorting, Searching, and Hashing**: Implement from scratch and benchmark seven sorting algorithms (Bubble, Selection, Insertion, Merge, Quick, Heap, Radix Sort), iterative/recursive Binary Search, and polynomial rolling hash tables with separate chaining and linear probing.
- **CO3: Non-Linear Hierarchical and Relational Structures**: Implement Binary Trees, Binary Search Trees (BST), self-balancing AVL Trees with all four rotation cases, arithmetic Expression Trees, and Graph representations (Adjacency Matrix, Adjacency List) with BFS, DFS, and Dijkstra's Shortest Path algorithm.

Crucially, the laboratory strictly prohibits the use of third-party algorithmic packages or built-in container abstractions (such as queue.Queue, collections.deque, heapq, bisect, or eval()) for core data structure mechanics, guaranteeing genuine algorithmic depth and authentic understanding.

5. Programming Environment and Tools

All programs across the seven laboratory units were developed, tested, and verified within a standardized POSIX Linux environment:

- **Operating System**: Linux (Ubuntu / Debian x86_64, POSIX-compliant shell environment).
- **Programming Language**: Python 3.8+ (Pure Standard Library: sys, typing, math, time).
- **Visualization Backend**: Matplotlib 3.3+ (utilized solely for rendering empirical benchmark curves and architectural diagrams).
- **Development Tools**: Antigravity IDE and Linux Bash Shell CLI.
- **Automated Self-Testing**: Every program features an integrated --test flag executing automated unit assertions to verify correctness, boundary edge cases, and exception handling.

6. Unit 1: Arrays and Matrices (CO1)

Overview: Unit 1 explores contiguous physical memory layouts, low-level element shifting mechanics, multidimensional row-major coordinate mappings, sparse matrix compression via 3-tuple coordinate lists, and a modular matrix arithmetic calculator.

Implemented Programs in Unit 1:

• **Program 1: 1D Array Operations** (array_operations.py): Implements low-level array behavior: insertion at index *k* with explicit rightward shifting; deletion with leftward shifting; linear search; and array rotation via the **Reversal Algorithm** (reversing subarrays *A[0..k-1]*, *A[k..n-1]*, then entire array in *O(n)* time and *O(1)* space).

• **Program 2: 2D Array Operations** (2d_array_operations.py): Models 2D grids using lists of lists. Implements cell access, coordinate deletion, 2D linear search, and 90-degree clockwise matrix rotation mapping cell *(r, c)* in an *M* × *N* matrix to *(c, M - 1 - r)* in an *N* × *M* matrix.

• **Program 3: Sparse Matrix Representation** (sparse_matrix.py): Implements the canonical **3-tuple (Coordinate List)** format. Header row stores [M, N, K] (rows, cols, non-zeros); subsequent *K* rows store [r, c, val]. Implements sparse transpose in *O(K log K)* and sparse addition via a two-pointer sorted merge in *O(K_A + K_B)* time, explicitly filtering zero cancellations.

• **Program 4: Matrix Calculator** (matrix_calculator.py): Menu-driven matrix suite supporting addition (*O(R·C)*), multiplication (*O(R_A·C_A·C_B)*), transpose, and determinant computation via recursive Laplace cofactor expansion (*O(n!)*) with singular matrix detection (*det(A) = 0*).

Sparse Matrix Space Optimization Mathematical Analysis:

For an *M* × *N* matrix with *K* non-zero elements and *S* bytes per scalar, Dense Storage requires *M* × *N* × *S* bytes, while Sparse 3-Tuple Storage requires *3* × *(K + 1)* × *S* bytes. The break-even threshold occurs when:

$$3(K + 1) < M \cdot N \implies K < (M \cdot N - 3) / 3$$

When non-zero elements constitute fewer than **33.3%** of matrix cells, sparse storage yields substantial memory savings (e.g. 98.47% reduction on a 100x100 matrix with 50 non-zeros).

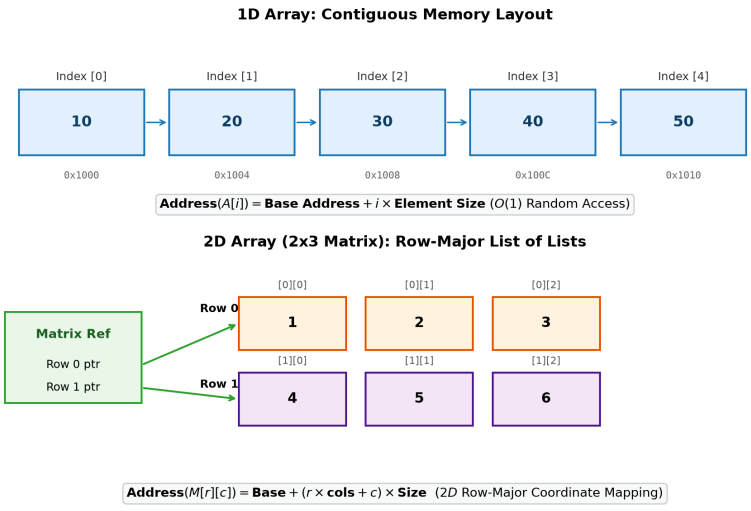


Figure 6.1: 1D Contiguous Memory Layout and 2D Row-Major Mapping

7. Unit 2: Sorting and Searching Algorithms (CO2)

Overview: Unit 2 provides an exhaustive investigation of comparison-based sorting, non-comparison digit sorting, logarithmic searching, hash collision resolution mechanisms, and practical record retrieval applications.

Implemented Programs in Unit 2:

• **Program 1: Bubble Sort** (bubble_sort.py): Optimized with early-termination flag swapped halting in $O(n)$ time on pre-sorted arrays.

• **Program 2: Selection Sort** (selection_sort.py): Locates unsorted minimum; strictly bounds memory writes to at most $n - 1$ swaps.

• **Program 3: Insertion Sort** (insertion_sort.py): Adaptive card-sorting logic shifting into sorted prefix; runs in $O(n)$ on nearly-sorted data.

• **Program 4: Merge Sort** (merge_sort.py): Divide-and-conquer bisection; stable $O(n \log n)$ sorting using an auxiliary merge buffer.

• **Program 5: Quick Sort** (quick_sort.py): In-place partitioning utilizing **median-of-three** pivot selection to prevent $O(n^2)$ worst-case degradation on sorted inputs.

• **Program 6: Heap Sort** (heap_sort.py): Constructs an in-place Binary Max-Heap in $O(n)$ time, followed by n extract-max passes in $O(n \log n)$ time and $O(1)$ auxiliary space.

• **Program 7: LSD Radix Sort** (radix_sort.py): Non-comparison sorting using stable Counting Sort passes across decimal digits in $O(d \cdot (n + b))$ time.

• **Program 8: Binary Search** (binary_search.py): Both iterative ($O(1)$ space) and recursive ($O(\log n)$ call-stack space) bisection algorithms.

• **Program 9: Performance Analysis** (sorting_comparison.py): Empirical benchmarking harness profiling all algorithms across Random, Sorted, and Reverse inputs of size $N = 100, 500, 1000$.

• **Program 10: Student Record Manager** (student_record_manager.py): Real-world database supporting multi-key sorting (Merge Sort for Roll Number & Marks; Quick Sort for Names) and Binary Search lookup.

• **Program 11: Search Engine Simulator** (search_engine.py): Dictionary keyword index comparing Hash Table (Separate Chaining & Linear Probing with polynomial rolling hash) against Sorted Array + Binary Search.

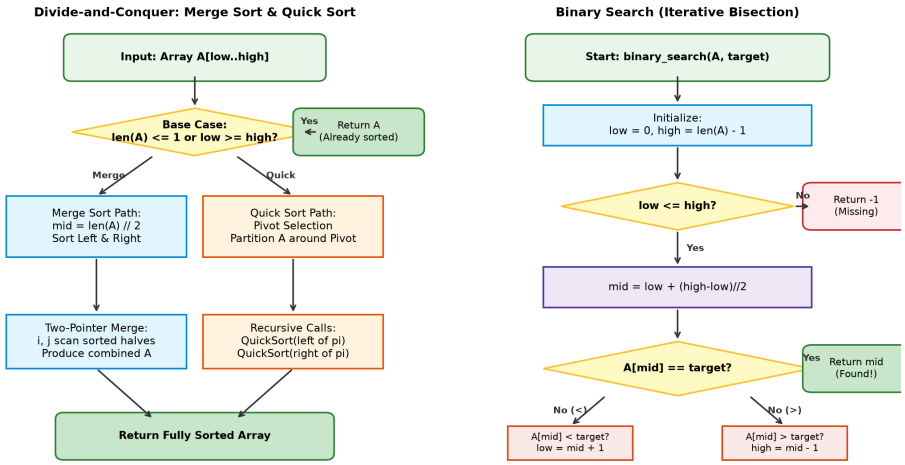


Figure 7.1: Algorithmic Logic for Merge Sort, Quick Sort, and Binary Search

8. Unit 3: Linked Lists (CO1)

Overview: Unit 3 investigates non-contiguous node-based linear data structures on the heap: Singly Linked Lists (SLL), Doubly Linked Lists (DLL), Circular Linked Lists (CLL), and applied systems for symbolic mathematics, document text buffers, and audio playlists.

Implemented Programs in Unit 3:

• **Program 1: Singly Linked List** (singly_linked_list.py): Maintains head pointer and _size. Supports zero-based positional insertion/deletion, linear search, in-place reversal via 3-pointer manipulation (prev, curr, next_node), and concatenation.

• **Program 2: Doubly Linked List** (doubly_linked_list.py): Maintains dual head and tail pointers and bidirectional prev / next links. Achieves *O(1)* boundary operations, enforces symmetry invariants (node.next.prev == node), and supports in-place reversal by swapping pointers.

• **Program 3: Circular Linked List** (circular_linked_list.py): Forms a closed-loop ring topology where tail.next == head. Employs cycle-safe termination conditions to traverse, search, reverse, and concatenate.

• **Program 4: Polynomial Operations** (polynomial_operations.py): Models single-variable polynomials as singly linked lists of (coeff, exp, next) sorted in descending exponent order. Implements polynomial addition via two-pointer merge in *O(n + m)*, term-by-term multiplication in *O(n·m)*, and Horner evaluation.

• **Program 5: Text Editor Buffer** (text_editor_buffer.py): Models document lines as a Doubly Linked List with a persistent cursor pointer, allowing *O(1)* line additions, deletions, and edits without shifting text arrays.

• **Program 6: Playlist Manager** (playlist_manager.py): Circular Doubly Linked List (CDLL) engine providing continuous next/previous song navigation, track insertion/removal, and infinite loop playback.

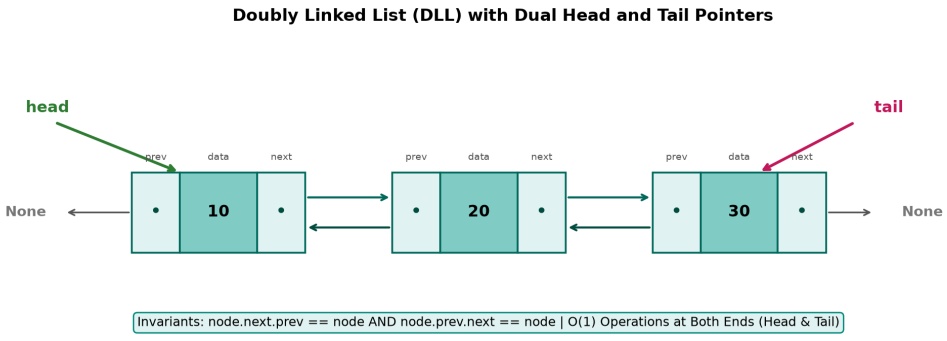


Figure 8.1: Doubly Linked List Architecture with Dual Pointers and Symmetry Invariant

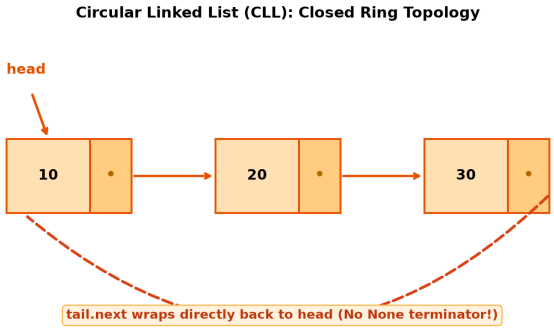


Figure 8.2: Circular Linked List Closed Ring Topology

9. Unit 4: Stacks (CO1)

Overview: Unit 4 focuses on the Last-In, First-Out (LIFO) operational discipline, comparing static array buffers against dynamic linked nodes, implementing the Shunting-Yard expression parsing algorithm, and engineering a dual-stack transaction rollback system.

Implemented Programs in Unit 4:

• **Program 1: Static Stack** (static_stack.py): Fixed-capacity array stack with _top index pointer; detects Stack Overflow and Stack Underflow in *O(1)* time.

• **Program 2: Dynamic Stack** (dynamic_stack.py): Singly linked node stack; unbounded capacity allocating memory dynamically on the heap in *O(1)* push/pop time.

• **Program 3: Infix to Postfix** (infix_to_postfix.py): **Shunting-Yard algorithm** parsing operator precedence (^ > */%// > +-) and left/right associativity with parenthesis grouping.

• **Program 4: Infix to Prefix** (infix_to_prefix.py): Converts infix to prefix notation by reversing tokens, inverting parenthesis direction, executing modified Shunting-Yard, and reversing output.

• **Program 5 & 6: Postfix & Prefix Evaluation** (postfix_evaluation.py / prefix_evaluation.py): Evaluates parenthesis-free expressions using an operand stack in linear *O(N)* time.

• **Program 7: Complete Expression Evaluator** (expression_evaluator.py): Standalone arithmetic evaluator with custom tokenizer supporting multi-digit numbers, floating points, and unary negation without eval().

• **Program 8: Undo/Redo System** (undo_redo.py): Command/Memento pattern utilizing two stacks (undo_stack and redo_stack); enforces history branching invalidation on new actions.

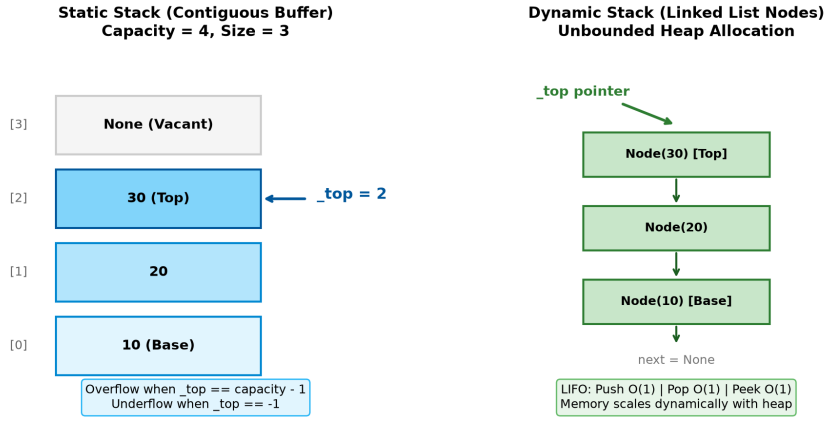


Figure 9.1: Static Array Buffer vs. Dynamic Linked Stack Memory Layouts

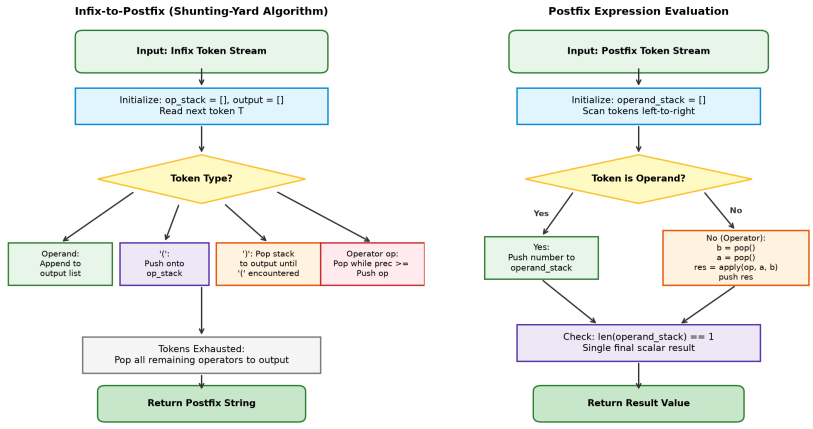


Figure 9.2: Shunting-Yard Infix Conversion and Postfix Evaluation Logic

10. Unit 5: Queues (CO1)

Overview: Unit 5 analyzes First-In, First-Out (FIFO) pipelines, linear array false overflow, circular queue modulo wraparound, double-ended deques, priority queue asymptotic trade-offs (array vs. binary heap), and discrete-event CPU scheduling simulations.

Implemented Programs in Unit 5:

• **Program 1: Static Queue** (static_queue.py): Fixed-size linear queue demonstrating the **false overflow defect** where new enqueues fail even if front slots are empty.

• **Program 2: Dynamic Queue** (dynamic_queue.py): Linked queue with dual front and rear pointers; unbounded FIFO scaling.

• **Program 3: Circular Queue** (circular_queue.py): Modulo arithmetic queue (ptr + 1) % cap that reuses vacated memory slots in *O(1)* time with an explicit _count variable.

• **Program 4: Deque** (deque.py): Circular array double-ended queue supporting *O(1)* insertions and deletions at both front and rear.

• **Program 5: Array Priority Queue** (array_priority_queue.py): Array implementation requiring *O(n)* scan or shift, featuring monotonic sequence counter for stable FIFO tie-breaking.

• **Program 6: Heap Priority Queue** (heap_priority_queue.py): Custom Binary Min-Heap with sift_up and sift_down, achieving optimal *O(log n)* push and pop operations.

• **Program 7: Customer Service Simulation** (customer_service.py): Multi-server simulation prioritizing VIP over Regular service queues.

• **Program 8: Process Scheduling Simulator** (process_scheduling.py): Preemptive **Round Robin CPU scheduler** with configurable time quantum *q*, computing Arrival Time (AT), Burst Time (BT), Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT).

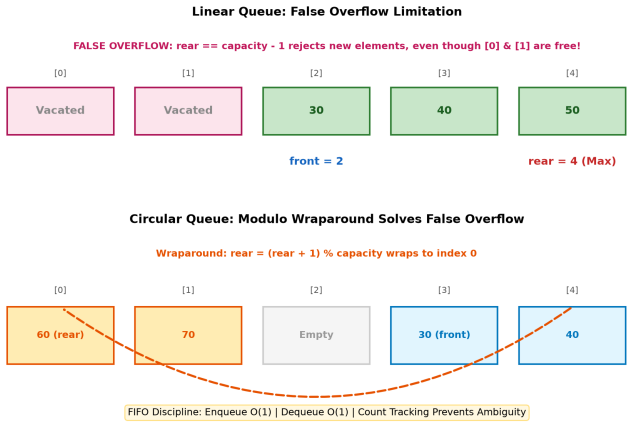


Figure 10.1: Linear Queue False Overflow vs. Circular Queue Modulo Wraparound

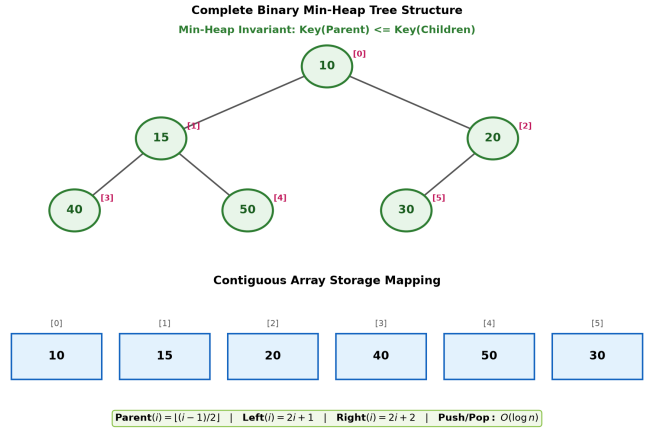


Figure 10.2: Binary Min-Heap Tree Structure and Contiguous Array Index Mapping

11. Unit 6: Trees (CO3)

Overview: Unit 6 explores non-linear hierarchical data structures: recursive and iterative binary tree traversals, Binary Search Trees (BST), self-balancing AVL Trees with 4 rotation primitives, arithmetic Expression Trees, and compiler symbol tables.

Implemented Programs in Unit 6:

• **Program 1 & 2: Recursive & Iterative Traversals** (recursive_traversals.py / iterative_traversals.py): Implements Preorder, Inorder, and Postorder traversals. Iterative version uses an explicit LIFO stack (including two-stack algorithm for Postorder) to bypass Python's 1000-frame recursion limit.

• **Program 3: Binary Search Tree (BST)** (binary_search_tree.py): Enforces ordering invariant *k_left < k_root < k_right*. Implements search, insert, min/max, recursive BST validation, and all three deletion cases (leaf node, single child, and two children replaced by inorder successor).

• **Program 4: Self-Balancing AVL Tree** (avl_tree.py): Maintains strict height balance factor $BF = h_L - h_R \in \{-1, 0, 1\}$. Implements all four rebalancing rotations: Left-Left (LL), Right-Right (RR), Left-Right (LR), and Right-Left (RL), guaranteeing *O(log n)* worst-case height $h \leq 1.44 \log(n)$.

• **Program 5: Expression Tree** (expression_tree.py): Constructs binary expression trees from postfix tokens using a node stack; performs recursive bottom-up evaluation and regenerates prefix, infix, and postfix formats.

• **Program 6: File System Simulator** (file_system.py): General *N*-ary tree simulating OS file directories with commands: mkdir, touch, ls, cd, rm, and pwd.

• **Program 7: Compiler Symbol Table** (symbol_table.py): BST-based identifier repository storing variable names, data types, scope depths, and source line numbers with duplicate detection.

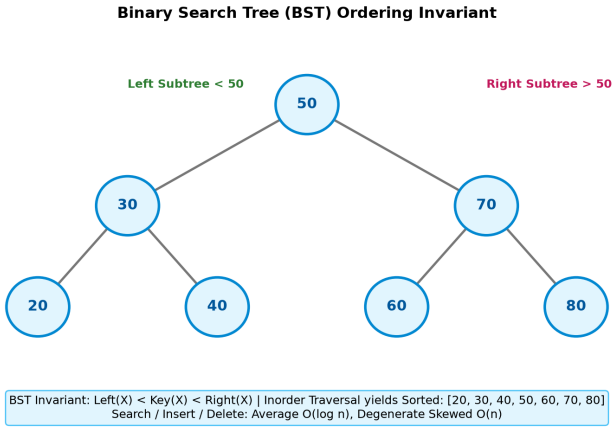


Figure 11.1: Binary Search Tree Invariant and Monotonic Inorder Sequence

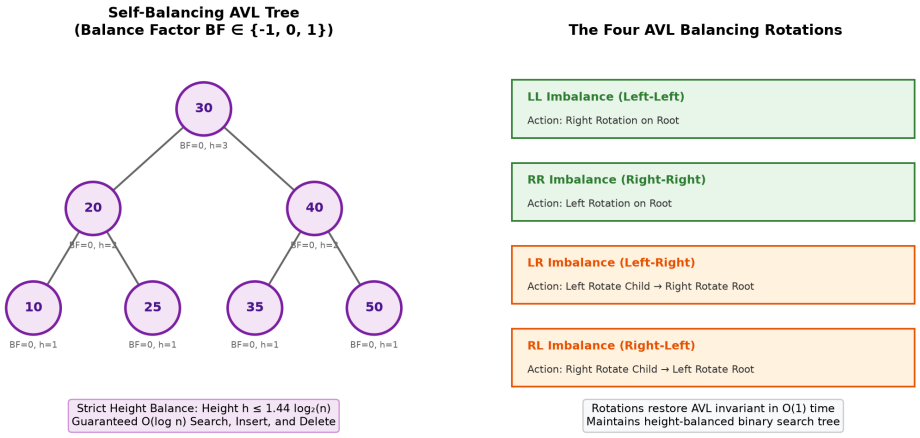


Figure 11.2: AVL Tree Balance Factors and the Four Rebalancing Rotations

12. Unit 7: Graphs (CO3)

Overview: Unit 7 covers relational graph architectures: Adjacency Matrix and Adjacency List representations, systematic BFS and DFS traversals, Dijkstra's Single-Source Shortest Path algorithm, and practical network routing and prerequisite systems.

Implemented Programs in Unit 7:

• **Program 1: Adjacency Matrix** (adjacency_matrix.py): 2D square matrix representation of size $|V| \times |V|$; provides *O(1)* edge existence queries at the cost of $O(V^2)$ memory.

• **Program 2: Adjacency List** (adjacency_list.py): Dynamic associative map of edge lists consuming optimal $O(V + E)$ memory for sparse networks.

• **Program 3: Breadth-First Search (BFS)** (bfs.py): Iterative level-by-level traversal using a custom manual FIFO queue; guarantees discovery of shortest unweighted paths.

• **Program 4 & 5: Recursive & Iterative DFS** (recursive_dfs.py / iterative_dfs.py): Depth-first backtracking traversal using call stack (recursive) and explicit LIFO stack (iterative, recursion-limit immune).

• **Program 6: Dijkstra's Algorithm** (dijkstra.py): Single-Source Shortest Path using a custom Binary Min-Heap Priority Queue; maintains the relaxation invariant $dist[v] = min(dist[v], dist[u] + w(u,v))$ in $O((V + E) \log V)$ time.

• **Program 7: Social Network Simulator** (social_network.py): Undirected social graph computing mutual friendships, 2-hop friend recommendations, and connected community clustering.

• **Program 8: 2D Maze Solver** (maze_solver.py): Models a spatial grid as an implicit graph and executes BFS to discover the optimal shortest escape path.

• **Program 9: Course Prerequisite Scheduler** (course_prerequisites.py): Directed Acyclic Graph (DAG) scheduling using **Kahn's Topological Sort Algorithm** with in-degree tracking and circular prerequisite cycle deadlock detection.

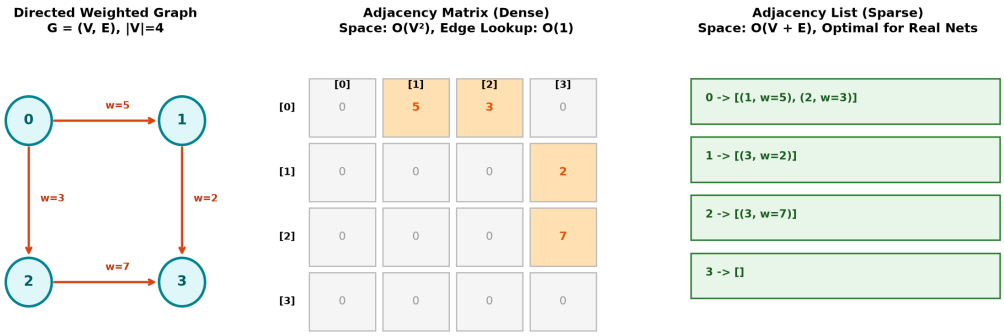


Figure 12.1: Directed Weighted Graph, Adjacency Matrix, and Adjacency List

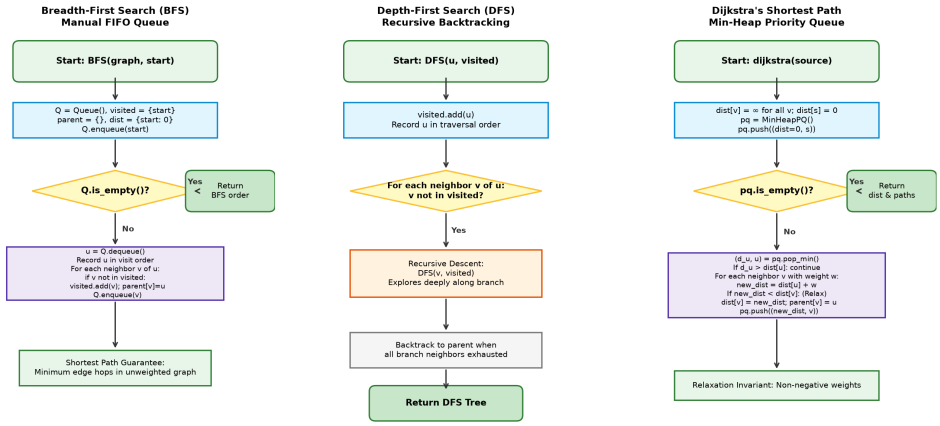


Figure 12.2: Algorithmic Logic for BFS, DFS, and Dijkstra's Shortest Path

13. Important Algorithms and Approaches

Across the seven laboratory units, several pervasive algorithmic paradigms and architectural patterns were implemented:

1. **Divide-and-Conquer**: Decomposes complex problems into smaller subproblems, solves them recursively, and combines the results. Exemplified by **Merge Sort** (bisecting lists into halves, recursively sorting, and merging in linear time) and **Quick Sort** (partitioning elements around a pivot value).
2. **Two-Pointer Sorted Merge Technique**: Simultaneously scans two sorted input streams to achieve linear $O(n + m)$ execution. Applied in Sparse Matrix Addition (merging row/column triples) and Polynomial Arithmetic (merging matching exponent terms).
3. **In-Place Memory Reversal Algorithm**: Reverses sequences by swapping symmetric elements from outside in. Applied in 1D Array Rotation (3 subarray reversals achieving $O(1)$ space), Singly/Doubly Linked List reversal, and Infix-to-Prefix conversion.
4. **Stack-Based Parsing (Shunting-Yard)**: Decouples operators and operands using precedence hierarchies and associativity rules to convert ambiguous infix grammar into deterministic postfix/prefix notation.
5. **Greedy Relaxation Invariant**: Systematically updates tentative shortest distances whenever an intermediate path offers lower cost: $dist[v] = min(dist[v], dist[u] + w(u,v))$. Governs **Dijkstra's Algorithm**.
6. **Cycle Detection & Topological Ordering (Kahn's Algorithm)**: Calculates vertex in-degrees in DAGs and iteratively evicts 0-in-degree nodes, verifying acyclic dependency integrity.
7. **Tree Self-Balancing via Rotations**: Restores logarithmic height bounds in AVL trees following insertion or deletion using local pointer reassignments in strict $O(1)$ time.

14. Complexity Analysis Master Summary

The following master reference table consolidates the theoretical Big-O time and auxiliary space complexities across all units:

Unit / Data Structure	Primary Operation	Best Case	Average Case	Worst Case	Aux. Space
Unit 1: 1D Array	Insertion / Deletion	`O(1) (tail)`	`O(n)`	`O(n) (head)`	`O(1)`
Unit 1: Sparse Matrix	Sparse Addition	`O(1)`	`O(K_A + K_B)`	`O(K_A + K_B)`	`O(K_A + K_B)`
Unit 2: Quick Sort	In-Place Sorting	`O(n log n)`	`O(n log n)`	`O(n²)`	`O(log n)`
Unit 2: Merge Sort	Stable Sorting	`O(n log n)`	`O(n log n)`	`O(n log n)`	`O(n)`
Unit 2: Binary Search	Iterative Search	`O(1)`	`O(log n)`	`O(log n)`	`O(1)`
Unit 3: Singly Linked List	Head Insert / Delete	`O(1)`	`O(1)`	`O(1)`	`O(1)`
Unit 3: Doubly Linked List	Tail Insert / Delete	`O(1)`	`O(1)`	`O(1)`	`O(1)`
Unit 4: Static/Dyn Stack	Push / Pop / Peek	`O(1)`	`O(1)`	`O(1)`	`O(1)`
Unit 4: Shunting-Yard	Infix to Postfix	`O(N)`	`O(N)`	`O(N)`	`O(N)`
Unit 5: Circular Queue	Enqueue / Dequeue	`O(1)`	`O(1)`	`O(1)`	`O(1)`
Unit 5: Min-Heap PQ	Push / Pop-Min	`O(1)`	`O(log n)`	`O(log n)`	`O(1)`
Unit 6: Binary Search Tree	Search / Insert / Delete	`O(1)`	`O(log n)`	`O(n)`	`O(h)`
Unit 6: AVL Tree	Search / Insert / Delete	`O(1)`	`O(log n)`	`O(log n)`	`O(log n)`
Unit 7: Graph BFS / DFS	Systematic Traversal	`O(V + E)`	`O(V + E)`	`O(V + E)`	`O(V)`
Unit 7: Dijkstra's Alg.	Shortest Path (Heap)	`O((V+E)log V)`	`O((V+E)log V)`	`O((V+E)log V)`	`O(V)`

15. Applications and Practical Use Cases

The data structures and algorithms developed in this laboratory power fundamental modern software systems:

- **Operating Systems & Kernel Scheduling**: Round Robin CPU dispatching utilizes circular queues to cycle processes fairly across time quantums. Priority queues backed by binary heaps manage kernel event alarms and real-time process priorities.
- **Compiler Design & Syntax Parsing**: Compilers transform human-readable arithmetic into machine code by tokenizing source streams, employing stacks in the Shunting-Yard algorithm to generate postfix tokens, building hierarchical Expression Trees, and maintaining identifier declarations in BST Symbol Tables.
- **Database Management Systems (DBMS)**: Relational database indices employ self-balancing tree structures (AVL trees, B-Trees) to guarantee *O(log n)* record retrieval. Secondary key lookups leverage polynomial hash tables for *O(1)* lookups.
- **Interactive Software & Buffer Editing**: Text editor line buffers utilize Doubly Linked Lists to allow localized line insertions and deletions without reallocating large memory blocks. Undo/Redo architectures use dual stacks to manage transaction rollback states.
- **Network Routing & Spatial Navigation**: Network routers use Dijkstra's Algorithm (OSPF protocol) to compute least-cost packet paths across weighted links. Web crawlers and autonomous maze navigation systems utilize Breadth-First Search (BFS) to find minimum-hop routes.
- **Dependency Management & Build Systems**: Package managers (npm, pip, Maven) and academic course schedulers represent dependency graphs as DAGs, employing Kahn's Topological Sort to detect circular deadlocks and establish valid build orders.

16. Challenges Encountered

Implementing advanced data structures from scratch in pure Python exposed several challenging edge cases and architectural dilemmas:

1. **Zero Cancellation in Sparse Matrix Addition**: When merging two non-zero entries at the same coordinate whose values sum to zero (e.g. 5 + (-5) = 0), the resulting entry must not be added to the triple representation, as storing zeros violates sparse compression. The implementation was revised to filter zero sums and decrement the non-zero element count dynamically.
2. **Rectangular Matrix 90-Degree Rotation**: Rotating an *M* × *N* rectangular matrix produces an *N* × *M* output. Mapping coordinates via Rotated[c][M - 1 - r] = Original[r][c] required careful boundary validation to prevent IndexError exceptions.
3. **Right-Associativity in Shunting-Yard**: Standard operators (+, -, *, /) are left-associative, popping stack operators with equal or greater precedence. Exponentiation (^) is right-associative (e.g. 2^3^2 = 2^(3^2) = 512). The parser was engineered to allow strictly greater precedence to pop right-associative operators.
4. **False Overflow in Linear Queues**: Observing how linear arrays trigger overflow when *rear = capacity - 1* despite empty front slots demonstrated the necessity of circular queues and modulo pointer arithmetic.
5. **BST Deletion with Two Children**: Removing a node with two children requires identifying its inorder successor (minimum key in right subtree), copying the successor's data, and recursively unlinking the duplicate successor node.
6. **AVL Rebalancing after Deletion**: Unlike insertion where at most one rotation restores global balance, AVL deletion can propagate balance factor imbalances up the entire tree height, requiring iterative ancestor balance checks up to the root.

17. Learning Outcomes

Completing the laboratory curriculum delivered profound technical and architectural competencies:

- **Contiguity vs Dynamic Allocation**: Mastered the fundamental trade-off between contiguous arrays (fast cache locality and *O(1)* indexing, slow *O(n)* shifting) and dynamic linked lists (flexible *O(1)* pointer insertions, zero shifting, higher memory overhead).
- **Asymptotic Discipline**: Developed instinctual fluency in evaluating theoretical Big-O complexity and distinguishing asymptotic growth from hardware wall-clock constants.
- **Defensive Engineering**: Cultivated rigorous programming discipline: validating matrix rectangularity, handling empty data structures, enforcing strict BST invariants, and testing boundary conditions.
- **Recursive to Iterative Transformation**: Mastered converting structural recursion into explicit stack-based iteration to overcome runtime stack limits in production environments.

18. Conclusion

This laboratory assignment achieved a comprehensive, from-scratch implementation and empirical evaluation of classical data structures and algorithms across seven intensive units. By strictly avoiding third-party libraries and built-in shortcuts, the repository preserves the authentic mechanics of pointer dereferencing, memory shifting, divide-and-conquer bisections, tree balancing rotations, and graph traversals. The resulting documentation, architectural diagrams, algorithm flowcharts, and complexity analyses form a complete, publication-grade academic reference.

19. References

1. Horowitz, E., Sahni, S., & Anderson-Freed, S. (2008). *Fundamentals of Data Structures in C* (2nd ed.). Silicon Press.
2. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.
3. Knuth, D. E. (1997). *The Art of Computer Programming, Volume 1: Fundamental Algorithms* (3rd ed.). Addison-Wesley.
4. Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley Professional.
5. Python Software Foundation. (2024). *Python 3.10+ Standard Library Documentation: Built-in Types and Data Structures*.
6. Strang, G. (2016). *Introduction to Linear Algebra* (5th ed.). Wellesley-Cambridge Press.